

Reporte de QA y Cobertura (Coverage)

1. Resumen de la Suite

Este reporte resume la estrategia de pruebas automatizadas implementada para la API FastAPI y te explica exactamente como leer el reporte HTML de cobertura que acabas de generar.

1. ESTRATEGIA DE PRUEBAS APLICADA

Se diseñaron 20 casos de prueba usando pytest y TestClient. Para garantizar que las pruebas sean rápidas, seguras y aisladas de tu base de datos real, se implementó un mecanismo de 'Mocking' que intercepta la conexión y la redirige a una base temporal en memoria/archivo.

Se validaron los siguientes escenarios para todos los endpoints:

- Happy Paths: Casos de uso ideales con datos válidos.
- Validaciones (422): Respuestas ante datos faltantes o de tipos incorrectos (gracias a Pydantic).
- Inyección SQL: Confirmación de que las sentencias SQLite preparadas son seguras.
- Tolerancia a Fallos: Simulación de caídas en base de datos para asegurar el retorno de errores HTTP 500 controlados.
- Idempotencia: Tolerancia a la doble ejecución de eliminaciones.

2. ¿Qué significa mi reporte de index.html?

2. TRADUCCIÓN DEL REPORTE HTML DE COBERTURA

El archivo index.html generado muestra qué porcentaje del código de tu main.py fue validado durante la ejecución de pytest. En tu caso particular, tu código de main.py se divide internamente en dos tecnologías distintas, lo cual explica por qué ves líneas rojas y verdes:

- LO QUE ESTÁ EN VERDE (Tus 10 Endpoints Web):

Si entras al detalle, verás que todas las líneas de abajo (de la línea 77 en adelante) correspondientes a la API Web (rutas con `@app.get`, `@app.post`, etc.) están totalmente cubiertas. Esto significa que el 100% de la lógica HTTP para gestionar autores, libros y préstamos está testeada y es confiable.

- LO QUE ESTÁ EN ROJO (Tu Servidor TCP):

Arriba (líneas 10 a 75), verás un bloque rojo grande que pertenece a la función `'handle_tcp_client'`. Esto es normal y esperado. Esa función levanta un servidor de Sockets puro asíncrono (asyncio TCP server) en el puerto 6061. Como nuestras pruebas fueron diseñadas usando el TestClient de FastAPI, su objetivo es únicamente golpear los endpoints HTTP. El servidor TCP nunca fue interactuado por los tests web, por eso aparece como "código no ejecutado".

- CONCLUSIÓN:

Tu API HTTP está robusta y cubierta. Para volver verde ese bloque superior rojo restante, sería necesario crear un segundo archivo de pruebas (`'test_tcp.py'`) que se conecte directamente al puerto 6061 mediante Sockets (usando `asyncio.open_connection`) para enviarle los comandos en formato JSON crudo.